

Samajh gaya. Pehle **product.json** bana raha hoon (20 products, **createdAt** MongoDB date format me, **kisi ka createdAt duplicate nahi**).

✓ **product.json (20 Products, Unique createdAt)**

```
141. [  
142.   {  
143.     "name": "Wireless Mouse",  
144.     "category": "electronics",  
145.     "price": 799,  
146.     "status": "active",  
147.     "tags": ["gadgets", "wireless", "mongodb"],  
148.     "rating": 4.3,  
149.     "stock": 120,  
150.     "createdAt": { "$date": "2024-01-01T10:00:00.000Z" }  
151.   },  
152.   {  
153.     "name": "Bluetooth Headphones",  
154.     "category": "electronics",  
155.     "price": 2499,  
156.     "status": "active",  
157.     "tags": ["audio", "bluetooth"],  
158.     "rating": 4.6,  
159.     "stock": 60,  
160.     "createdAt": { "$date": "2024-01-02T11:15:00.000Z" }  
161.   },  
162.   {  
163.     "name": "Gaming Keyboard",  
164.     "category": "electronics",  
165.     "price": 3499,
```

```
166.   "status": "active",
167.   "tags": ["gaming", "keyboard"],
168.   "rating": 4.7,
169.   "stock": 40,
170.   "createdAt": { "$date": "2024-01-03T09:30:00.000Z" }
171. },
172. {
173.   "name": "Notebook Pack",
174.   "category": "stationery",
175.   "price": 299,
176.   "status": "active",
177.   "tags": ["books", "notes"],
178.   "rating": 4.1,
179.   "stock": 200,
180.   "createdAt": { "$date": "2024-01-04T08:45:00.000Z" }
181. },
182. {
183.   "name": "Water Bottle",
184.   "category": "home",
185.   "price": 499,
186.   "status": "active",
187.   "tags": ["kitchen", "bottle"],
188.   "rating": 4.2,
189.   "stock": 150,
190.   "createdAt": { "$date": "2024-01-05T12:00:00.000Z" }
191. },
192. {
193.   "name": "Smart Watch",
194.   "category": "electronics",
195.   "price": 5999,
196.   "status": "active",
197.   "tags": ["watch", "fitness"],
198.   "rating": 4.5,
199.   "stock": 35,
200.   "createdAt": { "$date": "2024-01-06T14:20:00.000Z" }
```

```
201. },
202. {
203.   "name": "Office Chair",
204.   "category": "furniture",
205.   "price": 8999,
206.   "status": "active",
207.   "tags": ["chair", "office"],
208.   "rating": 4.4,
209.   "stock": 20,
210.   "createdAt": { "$date": "2024-01-07T16:10:00.000Z" }
211. },
212. {
213.   "name": "Study Table",
214.   "category": "furniture",
215.   "price": 6999,
216.   "status": "inactive",
217.   "tags": ["table", "study"],
218.   "rating": 4.0,
219.   "stock": 10,
220.   "createdAt": { "$date": "2024-01-08T10:40:00.000Z" }
221. },
222. {
223.   "name": "Yoga Mat",
224.   "category": "fitness",
225.   "price": 999,
226.   "status": "active",
227.   "tags": ["gym", "fitness"],
228.   "rating": 4.6,
229.   "stock": 90,
230.   "createdAt": { "$date": "2024-01-09T07:55:00.000Z" }
231. },
232. {
233.   "name": "Protein Powder",
234.   "category": "fitness",
235.   "price": 1899,
```

```
236.   "status": "active",
237.   "tags": ["health", "protein"],
238.   "rating": 4.8,
239.   "stock": 50,
240.   "createdAt": { "$date": "2024-01-10T18:30:00.000Z" }
241. },
242. {
243.   "name": "Cooking Oil 1L",
244.   "category": "groceries",
245.   "price": 299,
246.   "status": "active",
247.   "tags": ["kitchen", "oil"],
248.   "rating": 4.1,
249.   "stock": 180,
250.   "createdAt": { "$date": "2024-01-11T06:10:00.000Z" }
251. },
252. {
253.   "name": "Rice Bag 5kg",
254.   "category": "groceries",
255.   "price": 799,
256.   "status": "active",
257.   "tags": ["food", "rice"],
258.   "rating": 4.3,
259.   "stock": 70,
260.   "createdAt": { "$date": "2024-01-12T13:00:00.000Z" }
261. },
262. {
263.   "name": "Fiction Book",
264.   "category": "books",
265.   "price": 399,
266.   "status": "active",
267.   "tags": ["books", "reading"],
268.   "rating": 4.7,
269.   "stock": 100,
270.   "createdAt": { "$date": "2024-01-13T09:05:00.000Z" }
```

```
271. },
272. {
273.   "name": "History Book",
274.   "category": "books",
275.   "price": 499,
276.   "status": "inactive",
277.   "tags": ["books", "education"],
278.   "rating": 4.2,
279.   "stock": 45,
280.   "createdAt": { "$date": "2024-01-14T20:15:00.000Z" }
281. },
282. {
283.   "name": "Laptop Stand",
284.   "category": "electronics",
285.   "price": 1299,
286.   "status": "active",
287.   "tags": ["laptop", "office"],
288.   "rating": 4.4,
289.   "stock": 65,
290.   "createdAt": { "$date": "2024-01-15T15:50:00.000Z" }
291. },
292. {
293.   "name": "LED Desk Lamp",
294.   "category": "home",
295.   "price": 899,
296.   "status": "active",
297.   "tags": ["lamp", "light"],
298.   "rating": 4.5,
299.   "stock": 85,
300.   "createdAt": { "$date": "2024-01-16T11:25:00.000Z" }
301. },
302. {
303.   "name": "Mobile Charger",
304.   "category": "electronics",
305.   "price": 599,
```

```
306.   "status": "active",
307.   "tags": ["charger", "mobile"],
308.   "rating": 4.3,
309.   "stock": 140,
310.   "createdAt": { "$date": "2024-01-17T05:40:00.000Z" }
311. },
312. {
313.   "name": "Backpack",
314.   "category": "fashion",
315.   "price": 1499,
316.   "status": "active",
317.   "tags": ["bag", "travel"],
318.   "rating": 4.4,
319.   "stock": 75,
320.   "createdAt": { "$date": "2024-01-18T19:10:00.000Z" }
321. },
322. {
323.   "name": "Running Shoes",
324.   "category": "fashion",
325.   "price": 2999,
326.   "status": "active",
327.   "tags": ["shoes", "fitness"],
328.   "rating": 4.6,
329.   "stock": 55,
330.   "createdAt": { "$date": "2024-01-19T09:45:00.000Z" }
331. },
332. {
333.   "name": "Perfume",
334.   "category": "fashion",
335.   "price": 1999,
336.   "status": "inactive",
337.   "tags": ["fragrance", "style"],
338.   "rating": 4.2,
339.   "stock": 25,
340.   "createdAt": { "$date": "2024-01-20T21:30:00.000Z" }
```

341. }

342.]

MongoDB Collections Show Command

343. show collections

MongoDB Functions (Tumhare Required)

1) Limit result to show only 10 documents?

344. db.products.find().limit(10)

2) Sort documents ascending order based on name?

345. db.products.find().sort({ name: 1 })

3) Sort documents descending order based on createdAt?

346. `db.products.find().sort({ createdAt: -1 })`

4) Skip first 5 and display next 10?

347. `db.products.find().skip(5).limit(10)`

5) Count total documents in a collection?

348. `db.products.countDocuments()`

6) Count documents where status is "active"?

349. `db.products.countDocuments({ status: "active" })`

7) Update single document where email is "test@example.com" set verified true?

```
350. db.users.updateOne(  
351.   { email: "test@example.com" },  
352.   { $set: { verified: true } }  
353. )
```

8) Update all documents where category is "books" add discount 15%?

```
354. db.products.updateMany(  
355.   { category: "books" },  
356.   { $set: { discount: "15%" } }  
357. )
```

9) Delete one document where status is "inactive"?

```
358. db.products.deleteOne({ status: "inactive" })
```

10) Delete all documents where expired is true?

```
359. db.products.deleteMany({ expired: true })
```

11) Find first 5 documents sorted by price descending?

360. `db.products.find().sort({ price: -1 }).limit(5)`

12) Find only name and email fields for all users?

361. `db.users.find({}, { name: 1, email: 1, _id: 0 })`

13) Find documents where age > 25?

362. `db.users.find({ age: { $gt: 25 } })`

14) Find documents where price between 100 and 500?

363. `db.products.find({ price: { $gte: 100, $lte: 500 } })`

15) Find all documents where tags contain "mongodb"?

364. `db.products.find({ tags: "mongodb" })`

16) Update only lastLogin field for username "john_doe"?

```
365. db.users.updateOne(  
366.   { username: "john_doe" },  
367.   { $set: { lastLogin: new Date() } }  
368. )
```

17) Remove specific field discount from all documents?

```
369. db.products.updateMany(  
370.   {},  
371.   { $unset: { discount: "" } }  
372. )
```

18) Total number of documents where rating > 4.5?

```
373. db.products.countDocuments({ rating: { $gt: 4.5 } })
```

19) Upsert (update if exists else insert)?

```
374. db.users.updateOne(  
375.   { email: "newuser@example.com" },  
376.   { $set: { name: "New User", verified: false } },
```

```
377. { upsert: true }  
378. )
```

20) Delete oldest document based on createdAt?

```
379. db.products.deleteOne(  
380.   { _id: db.products.find().sort({ createdAt: 1  
      }).limit(1).toArray()[0]._id }  
381. )
```

⚠ Ye Mongo shell me chalega but thoda unsafe hai. Better approach:

```
382. let oldest = db.products.find().sort({ createdAt: 1  
      }).limit(1).toArray()  
383. db.products.deleteOne({ _id: oldest[0]._id })
```

✅ Extra 100+ MongoDB Questions (Hinglish Notes Style)

Basic Find Queries

1. Kaise find karoge saare documents?
2. Kaise sirf ek document find karoge?
3. Kaise find karoge jaha category = "electronics" ho?
4. Kaise find karoge jaha price 1000 se zyada ho?
5. Kaise find karoge jaha price 500 se kam ho?

6. Kaise find karoge jaha rating exactly 4.5 ho?
7. Kaise find karoge jaha stock 0 ho?
8. Kaise find karoge jaha status active nahi ho?
9. Kaise find karoge jaha category books OR fashion ho?
10. Kaise find karoge jaha category books AND status active ho?

Projection (Select Fields)

11. Kaise sirf name field show karoge?
12. Kaise name aur price show karoge?
13. Kaise _id hide karoge?
14. Kaise sirf category aur stock show karoge?
15. Kaise sirf createdAt show karoge?
16. Kaise tags field show karoge but baaki hide karoge?
17. Kaise multiple fields exclude karoge?
18. Kaise email field include karoge but password exclude karoge?
19. Kaise nested field show karoge?
20. Kaise nested field hide karoge?

Sorting

21. Kaise price ascending sort karoge?
22. Kaise price descending sort karoge?
23. Kaise rating descending sort karoge?
24. Kaise createdAt ascending sort karoge?
25. Kaise createdAt descending sort karoge?
26. Kaise name ascending sort karoge?
27. Kaise name descending sort karoge?
28. Kaise category wise sorting karoge?
29. Kaise sort by multiple fields (category then price)?
30. Kaise sort by status then createdAt?

Limit and Skip

31. Kaise top 3 expensive products nikaloge?

32. Kaise top 10 cheapest products nikaloge?
33. Kaise page 2 ke results nikaloge?
34. Kaise page 3 ke results nikaloge?
35. Kaise skip 20 documents then show 10?
36. Kaise skip 0 then show 5?
37. Kaise last 5 documents nikaloge createdAt ke according?
38. Kaise random pagination implement karoge?
39. Kaise first 50 documents fetch karoge?
40. Kaise limit 1 use karke single latest product nikaloge?

Count Queries

41. Kaise total users count karoge?
42. Kaise total products count karoge?
43. Kaise count karoge jaha stock > 50 ho?
44. Kaise count karoge jaha price between 2000-5000 ho?
45. Kaise count karoge jaha category = "books" ho?
46. Kaise count karoge jaha status inactive ho?
47. Kaise count karoge jaha rating >= 4.0 ho?
48. Kaise count karoge jaha tags me "fitness" ho?
49. Kaise count karoge jaha discount field exist karta ho?
50. Kaise count karoge jaha discount field missing ho?

Update Queries

51. Kaise ek document me price update karoge?
52. Kaise multiple documents me price update karoge?
53. Kaise ek product ka stock reduce karoge by 1?
54. Kaise stock increase karoge by 10?
55. Kaise rating update karoge?
56. Kaise ek user ka password update karoge?
57. Kaise ek field add karoge sab documents me?
58. Kaise new boolean field add karoge?
59. Kaise update karoge but sirf active products me?
60. Kaise update karoge jaha category electronics ho?

\$set and \$unset

61. Kaise field add karoge using \$set?
62. Kaise field remove karoge using \$unset?
63. Kaise discount remove karoge only books category me?
64. Kaise verified false set karoge sab users me?
65. Kaise lastLogin update karoge current date se?
66. Kaise createdAt update karoge manually?
67. Kaise status inactive set karoge jaha stock = 0 ho?
68. Kaise tags array update karoge?
69. Kaise address field update karoge?
70. Kaise nested field update karoge?

\$inc, \$push, \$pull

71. Kaise price increase karoge by 100 using \$inc?
72. Kaise stock decrease karoge using \$inc?
73. Kaise tags me new tag push karoge?
74. Kaise tags me multiple values add karoge?
75. Kaise tags se ek value remove karoge?
76. Kaise array empty karoge?
77. Kaise array me duplicate avoid karoge (\$addToSet)?
78. Kaise comments array me object push karoge?
79. Kaise last element remove karoge array se?
80. Kaise first element remove karoge array se?

Delete Queries

81. Kaise ek product delete karoge name ke basis pe?
82. Kaise multiple products delete karoge category basis pe?
83. Kaise inactive products delete karoge?
84. Kaise delete karoge jaha rating < 3 ho?
85. Kaise delete karoge jaha stock = 0 ho?
86. Kaise delete karoge jaha expired = true ho?
87. Kaise sab documents delete karoge collection se?

- 88. Kaise sirf ek oldest product delete karoge?
- 89. Kaise latest product delete karoge?
- 90. Kaise delete karoge jaha createdAt 2023 se pehle ka ho?

Query Operators

- 91. Kaise \$gt use karoge?
- 92. Kaise \$lt use karoge?
- 93. Kaise \$gte use karoge?
- 94. Kaise \$lte use karoge?
- 95. Kaise \$ne use karoge?
- 96. Kaise \$in use karoge?
- 97. Kaise \$nin use karoge?
- 98. Kaise \$or use karoge?
- 99. Kaise \$and use karoge?
- 100. Kaise \$not use karoge?

Regex / Search

- 101. Kaise name search karoge starting with "W"?
- 102. Kaise name search karoge ending with "Book"?
- 103. Kaise case-insensitive search karoge?
- 104. Kaise email me "@gmail.com" wale users find karoge?
- 105. Kaise partial word match karoge?
- 106. Kaise product search karoge "shoe" keyword se?
- 107. Kaise regex search me multiple patterns check karoge?
- 108. Kaise exact match vs partial match implement karoge?
- 109. Kaise regex se category filter karoge?
- 110. Kaise regex me ignore-case flag lagate ho?

Date Queries

- 111. Kaise find karoge documents created after 2024-01-10?
- 112. Kaise find karoge documents created before 2024-01-05?
- 113. Kaise last 7 days ke documents find karoge?

- 114. Kaise current month ke documents find karoge?
- 115. Kaise createdAt sort karke newest nikaloge?
- 116. Kaise createdAt sort karke oldest nikaloge?
- 117. Kaise date range me find karoge?
- 118. Kaise date field update karoge?
- 119. Kaise date field remove karoge?
- 120. Kaise ISODate format me query likhte ho?

Advanced Find

- 121. Kaise find karoge jaha field exist karta ho?
- 122. Kaise find karoge jaha field missing ho?
- 123. Kaise find karoge jaha stock type number ho?
- 124. Kaise find karoge jaha price string me stored ho (bad data)?
- 125. Kaise null values wale documents find karoge?
- 126. Kaise empty string wale documents find karoge?
- 127. Kaise array length check karoge?
- 128. Kaise tags me 2 values required ho?
- 129. Kaise embedded object match karoge?
- 130. Kaise nested array match karoge?

Upsert + Replace

- 131. Kaise upsert karoge username basis pe?
- 132. Kaise replaceOne use karoge?
- 133. Kaise updateOne vs replaceOne difference explain karoge?
- 134. Kaise updateMany me upsert possible hai ya nahi?
- 135. Kaise ensure karoge duplicate emails insert na ho?

Index Related Questions

- 136. Kaise index create karoge name field pe?
- 137. Kaise compound index banate ho?
- 138. Kaise unique index banate ho email pe?
- 139. Kaise index drop karoge?

140. Kaise check karoge indexes?

Aggregation Basics

141. Kaise group by category karoge?

142. Kaise category wise count nikaloge?

143. Kaise average price nikaloge?

144. Kaise max price nikaloge?

145. Kaise min price nikaloge?

146. Kaise total stock sum karoge?

147. Kaise sort aggregation results?

148. Kaise limit aggregation results?

149. Kaise match + group use karoge?

150. Kaise project stage use karoge?



Required Functions Examples (Short Notes)

sort()

384. `db.products.find().sort({ price: -1 })`

skip()

385. `db.products.find().skip(5)`

countDocuments({query})

386. `db.products.countDocuments({ status: "active" })`

updateOne / updateMany

387. db.products.updateOne({ name: "Perfume" }, { \$set: { price: 1799 } })

388.

389. db.products.updateMany({ category: "books" }, { \$set: { discount: "15%" } })

deleteOne / deleteMany

390. db.products.deleteOne({ status: "inactive" })

391.

392. db.products.deleteMany({ expired: true })